skillsとspecsを使って開発を進める手順書

初学者向け / Codex・Claude Code 両対応

- 入力として使うもの
  - `skills/`
  - `C:\Users\username\work\minimal-skills-pack\specs`
- 目的
  - 仕様を整理してから、1 タスクずつ安全に開発を進める

この資料でやること

1. 作業用 project を作る
2. `specs` から使う仕様書をコピーする
3. `skills` をその project で使えるようにする
4. AI を起動する
5. `docs/input-specs/汎用サービス開発仕様プロンプト.md` を参考に開発要件を決める
6. `docs/input-specs/汎用サービス開発仕様プロンプト.md` を修正させる
7. 最初は実装させず、`SPEC.md` とタスク分解だけ作らせる
8. 1 タスクずつ実装させる
9. ずれたときに立て直す

先に知っておくこと

- `skills` は作業のやり方の指示書
- `specs` は今回作るアプリの要求仕様
- 最初から「全部作って」と頼むと失敗しやすい
- **先に計画、あとで実装** の順が安全
- **1 タスクずつ進める** と壊れた場所を追いやすい

おすすめの構成

- skills 集の repository
  - `C:\Users\username\work\minimal-skills-pack`
- 実際にアプリを作る作業用 project
  - 例: `C:\Users\username\work\my-first-app`

skills 集そのものと、実際の開発先は分けてください。
このほうが誤って skills 集を壊しにくくなります。

Step 1. 作業用 project を作る

PowerShell

```
New-Item -ItemType Directory -Force -Path C:\Users\username\work\my-first-app | Out-Null
Set-Location C:\Users\username\work\my-first-app
```

bash

```
mkdir -p /c/Users/username/work/my-first-app
cd /c/Users/username/work/my-first-app
```

`bash` は Git Bash 前提です。
WSL の場合は `/mnt/c/Users/...` に読み替えてください。

Step 2. spec を project にコピーする

初学者は、`specs` を直接参照するより、**使う spec を project にコピー** したほうが安全です。

- AI に読ませるファイルを明確にしやすい
- project の中だけ見れば必要情報がそろう
- 後で spec を project 側で調整しやすい

使う例:

- `C:\Users\username\work\minimal-skills-pack\specs\汎用サービス開発仕様プロンプト.md`

Step 2. spec をコピーするコマンド

PowerShell

```
Set-Location C:\Users\username\work\my-first-app
New-Item -ItemType Directory -Force -Path .\docs\input-specs | Out-Null
Copy-Item `
  -Path "C:\Users\username\work\minimal-skills-pack\specs\汎用サービス開発仕様プロンプト.md" `
  -Destination .\docs\input-specs\
```

bash

```
cd /c/Users/username/work/my-first-app
mkdir -p ./docs/input-specs
cp "/c/Users/username/work/minimal-skills-pack/specs/汎用サービス開発仕様プロンプト.md" ./docs/input-specs/
```

Step 2. コピー後に確認すること

project 側に次のファイルがある状態にしてください。

- `docs/input-specs/汎用サービス開発仕様プロンプト.md`

このあと AI には、**project 内のこのファイル** を読ませます。

Step 3. skills を project で使えるようにする

使う AI に応じて、skills の置き場所が変わります。

- Codex
  - `./.agents/skills`
  - `./.agents/references`
- Claude Code
  - `./.claude/skills`
  - `./.claude/references`

詳しいコマンドは次の手順書を見てください。

- Codexでskillsを使う手順書.md
- ClaudeCodeでskillsを使う手順書.md

Step 4. AI を起動する

Codex

```
Set-Location C:\Users\username\work\my-first-app
codex
```

```
cd /c/Users/username/work/my-first-app
codex
```

Claude Code

```
Set-Location C:\Users\username\work\my-first-app
claude
```

```
cd /c/Users/username/work/my-first-app
claude
```

Step 5. 最初は実装させない

最初の入力でやらせることは 5 つです。

1. `docs/input-specs/汎用サービス開発仕様プロンプト.md` を読む
2. その内容を参考に、今回の project の開発要件を決める
3. 決めた内容を反映して `docs/input-specs/汎用サービス開発仕様プロンプト.md` を修正する
4. 必要な skill を選ぶ
5. `SPEC.md` と `tasks/plan.md` と `tasks/todo.md` を作る

まだコード実装は始めさせません。

Step 5. 最初の入力文

```
この project では、利用可能な skills を使いながら開発を進めてください。

まず次のファイルを読んでください。
- docs/input-specs/汎用サービス開発仕様プロンプト.md

作業の進め方は次の条件に従ってください。
- docs/input-specs/汎用サービス開発仕様プロンプト.md を参考にして、今回の project の開発要件を決める
- 決めた開発要件を docs/input-specs/汎用サービス開発仕様プロンプト.md に反映して修正する
- 最初はいきなり実装しない
- まず仕様を整理して SPEC.md を作る
- 次に tasks/plan.md と tasks/todo.md を作る
- タスクは大きすぎる単位にせず、縦切りで小さく分ける
- 最初の 3 タスクは特に小さく分ける
- どの skill を使うかも明示する
```

Step 5. 最初の入力文の続き

```
出力してほしいもの:
- 修正後の docs/input-specs/汎用サービス開発仕様プロンプト.md
- SPEC.md
- tasks/plan.md
- tasks/todo.md

この入力では、まだコード実装は始めないでください。
```

Step 6. 最初の返答で確認すること

次の 4 つが作られているか見ます。

- `docs/input-specs/汎用サービス開発仕様プロンプト.md`
- `SPEC.md`
- `tasks/plan.md`
- `tasks/todo.md`

確認ポイント:

- `docs/input-specs/汎用サービス開発仕様プロンプト.md`
  - 今回の project に合わせて要件が調整されているか
  - 不要な要件や不足要件が整理されているか
- `SPEC.md`
  - 何を作るかが明確か
  - スコープ内 / 外が分かれているか
- `tasks/plan.md`
  - 順番が無理なく並んでいるか
- `tasks/todo.md`
  - 1 タスクが大きすぎないか

Step 7. タスクが大きすぎるとき

大きいと感じたら、実装に進む前に細かくさせます。

```
tasks/todo.md のタスクがまだ大きいです。

次の条件で分割し直してください。
- 1 タスクごとに完了確認しやすい大きさにする
- できるだけ 1 タスクで 1 つの画面、1 つの API、1 つの責務に寄せる
- 各タスクの完了条件を書く
- 依存関係が分かるようにする

まだ実装は始めないでください。
```

Step 8. 1 タスク目だけ実装させる

計画に納得したら、**最初の 1 タスクだけ** 実装させます。

```
では tasks/todo.md の 1 タスク目だけ実装してください。

条件:
- 1 タスク目以外は進めない
- 変更したファイルを最後にまとめる
- 実装後に typecheck と build を実行する
- 失敗した場合は、何が原因かを整理する
```

Step 8. 実装後に報告させること

```
最後に次を報告してください。
- 何を変更したか
- どこまで終わったか
- 実行した確認項目
- 次に進む前に人間が見るべき点
```

1 タスクずつ終わらせることで、問題の切り分けがしやすくなります。

Step 9. 2 タスク目以降も同じ形

毎回「次の 1 タスクだけ」と伝えます。

```
では tasks/todo.md の 2 タスク目だけ実装してください。
1 タスクずつ進めたいので、他のタスクには手を出さないでください。
実装後は typecheck と build も実行してください。
```

3 タスク目、4 タスク目も同じ形で進めます。

Step 10. 会話が長くなったら整理して再開

長いセッションでは前提がぶれやすくなります。

```
ここまでの状況を整理してください。

確認したいこと:
- 完了済みタスク
- 未着手タスク
- 今の実装が spec から外れていないか
- 次にやるべき 1 タスク
```

Step 10. 再開用の入力

```
そのうえで、次は tasks/todo.md の次の 1 タスクだけ進めてください。
```

この一言を付けると、次の作業に戻りやすくなります。

Step 11. spec と実装がずれてきたとき

違和感が出たら、そのまま進めず確認させます。

```
いまの実装が spec からずれていないか確認してください。

比較対象:
- docs/input-specs/汎用サービス開発仕様プロンプト.md
- SPEC.md

やってほしいこと:
- ずれている点を列挙する
- 重大なものから順に出す
- まだ修正は始めず、修正方針だけ出す
```

Step 12. 最後の仕上げ

主要タスクが終わったら、最後に次をまとめて確認させます。

- `tasks/todo.md` に未完了が残っていないか
- `SPEC.md` の要件を満たしているか
- `npm run typecheck` が通るか
- `npm run build` が通るか
- ドキュメント不足がないか

Step 12. 最終確認の入力文

```
最終確認をしてください。

確認対象:
- SPEC.md
- tasks/todo.md
- 現在の実装

やってほしいこと:
- 未完了項目を出す
- 受け入れ条件に未達があれば出す
- typecheck と build を実行する
- ドキュメント不足があれば出す
```

初学者が迷いやすいポイント

- 最初から全部作らせる
  - まずは計画だけにする
- 1 回で複数タスクを進めさせる
  - 毎回 1 タスクだけ
- spec を project 外のまま使う
  - `docs/input-specs/` にコピーしてから使う
- skills を有効化しない
  - 先に `.agents` または `.claude` を整える
- 実装後の確認を省く
  - 毎回 `typecheck` と `build`

迷ったときの最小セット

1. 作業用 project を作る
2. spec を `docs/input-specs/` にコピーする
3. skills を有効にする
4. AI に `SPEC.md` と `tasks/plan.md` と `tasks/todo.md` を作らせる
5. 1 タスクずつ実装させる
6. 毎回 `typecheck` と `build` を通す

最初の入力文だけ見たい人向け

```
この project では、利用可能な skills を使いながら開発を進めてください。
まず docs/input-specs/汎用サービス開発仕様プロンプト.md を読んでください。

最初にやること:
- docs/input-specs/汎用サービス開発仕様プロンプト.md を参考にして開発要件を決める
- 決めた内容で docs/input-specs/汎用サービス開発仕様プロンプト.md を修正する
- SPEC.md を作る
- tasks/plan.md を作る
- tasks/todo.md を作る
- タスクは小さく縦切りで分ける

この入力では、まだ実装しないでください。
```

まとめ

- skills はやり方、specs は仕様
- 最初に input-specs を project 向けに直す
- 最初は **計画だけ**
- 実装は **1 タスクずつ**
- 毎回 `typecheck` と `build`
- ずれたら spec と `SPEC.md` に戻って確認する